COMPLEXITY ANALYSIS REPORT

Soccer Team Strategy Simulator

APM11A1 - Code Challenge 1 (CC1)

TEAM MEMBERS

Student Number	Name	Role in Complexity Analysis
221152725	Mongiwethu Eddy Ncube	Lead Developer, overall coordination and final review
220115085	Sandile Sibeko	Documentation and formatting of analysis report
220068905	Keamogetse Selebano	Linear search and binary search implementation and analysis
220122253	Ndzulamo Michelle Yingwani	Complexity comparison tables and formatting
220080694	Hlonipho Nersely Bila	Space complexity breakdown and memory analysis
220061777	Zizile Ezona Mbanqi	Testing and validation of complexity claims
219027546	Bongane Sithole	Bubble sort and quick sort implementation and analysis

COMPLEXITY ANALYSIS REPORT

Soccer Team Strategy Simulator

APM11A1 - Code Challenge 1 (CC1)

Date: 2026-04-18

Submitted by: Group F

---

1. INTRODUCTION

This report provides a detailed complexity analysis of the search and sorting algorithms implemented in our Soccer Team Strategy Simulator. The algorithms analysed include linear search, binary search, bubble sort, and quick sort. While our application currently manages a maximum of eleven players per team, understanding how these algorithms would perform with larger datasets is essential for demonstrating our knowledge of algorithmic efficiency. This document outlines the time complexity, space complexity, and practical performance considerations for each algorithm.

2. SEARCH ALGORITHMS

2.1 Linear Search

Implementation Overview

The linear search algorithm iterates through the player list sequentially until a matching player name is found or the end of the list is reached. This approach makes no assumptions about the order of the data.

Time Complexity Analysis

Case	Complexity	Explanation
Best Case	O(1)	Target player is the first element in the list
Worst Case	O(n)	Target player is the last element or not present
Average Case	O(n)	Approximately n/2 comparisons are required

Space Complexity Analysis

The space complexity is O(1) as the algorithm uses only a constant amount of additional memory regardless of input size. No auxiliary data structures are created during execution.

Performance Example with Eleven Players

Scenario	Number of Comparisons
Best Case	1
Worst Case	11
Average Case	5 to 6

Code Implementation

```
Player* linearSearchByName(const
string& name) {
    for (auto& p : players) {
        if (p->getName() == name) {
            return p;
        }
    }
    return nullptr;
}
```

Practical Application

Linear search is appropriate for our simulator because the team size is small and players are frequently added and removed, making it inefficient to maintain a sorted structure solely for search operations.

2.2 Binary Search

Implementation Overview

Binary search requires the player list to be sorted by jersey number before execution. The algorithm repeatedly divides the search interval in half, comparing the target value with the middle element and eliminating half of the remaining elements with each iteration.

Time Complexity Analysis

Case	Complexity	Explanation
Best Case	O(1)	Target is found at the middle position on first attempt
Worst Case	O(log n)	Search space is halved until only one element remains
Average Case	O(log n)	Approximately $\log_2(n)$ comparisons are required

Space Complexity Analysis

The iterative implementation achieves O(1) space complexity as only a few variables (left, right, mid) are needed regardless of input size.

Performance Example with Eleven Players

With eleven players, the maximum number of comparisons required is four, because $2^4 = 16$ which exceeds 11. This represents a significant improvement over linear search.

Important Consideration

Binary search requires the data to be sorted beforehand. In our implementation, we call quick sort before performing binary search, which adds an additional O(n log n) preprocessing cost.

Code Implementation

```
Player* binarySearchByNumber(int number) {
    quickSortByNumber(0, players.size() - 1);  //
Pre-sorting required

    int left = 0, right = players.size() - 1;
    while (left <= right) {
```

```
        int mid = left + (right - left) / 2;
        if (players[mid]->getNumber() == number) {
            return players[mid];
        }
        else if (players[mid]->getNumber() <
 number) {
            left = mid + 1;
        }
        else {
            right = mid - 1;
        }
    }
    return nullptr;
}
```

Mathematical Proof

Let T(n) represent the number of comparisons for n elements. After each comparison, the search space reduces to n/2. After k comparisons, the remaining elements are $n/2^k$. When $n/2^k = 1$, we have $k = \log_2(n)$. Therefore, the time complexity is O(log n).

3. SORTING ALGORITHMS

3.1 Bubble Sort

Implementation Overview

Bubble sort works by repeatedly stepping through the player list, comparing adjacent elements, and swapping them if they are in the wrong order. Larger values "bubble up" to their correct positions after each pass. We implemented this algorithm to sort players by skill rating in descending order.

Time Complexity Analysis

Case	Complexity	Explanation
Best Case	O(n)	Array is already sorted; only one pass required
Worst Case	$O(n^2)$	Array is in reverse order; maximum swaps required
Average Case	$O(n^2)$	Approximately $n^2/4$ comparisons

Space Complexity Analysis

Bubble sort operates in-place with O(1) space complexity. No additional arrays are created; all swaps occur within the existing data structure.

Performance Example with Eleven Players

The maximum number of comparisons for eleven players is 55, calculated as $n(n-1)/2 = (11 \times 10)/2 = 55$. The maximum number of swaps is also 55 in the worst-case scenario.

Code Implementation

```
void bubbleSortBySkill() {
    int n = players.size();
    for (int i = 0; i < n - 1; i++) {
```

```
        for (int j = 0; j < n - i - 1; j++) {
            if (players[j]->getSkill() < players[j + 1]->getSkill()) {
                swap(players[j], players[j + 1]);  // Descending order
            }
        }
    }
}
```

Mathematical Proof

The outer loop executes (n-1) times. The inner loop executes (n-i-1) times for each outer iteration. The total number of comparisons is the sum of the arithmetic series:

$(n-1) + (n-2) + \ldots + 2 + 1 = n(n-1)/2$

Therefore, the time complexity is $O(n^2)$.

Educational Justification

While bubble sort is not the most efficient algorithm, we included it because the rubric specifically required it. Additionally, for a fixed maximum of eleven players, 55 operations execute in negligible time (less than one millisecond on modern hardware).

3.2 Quick Sort

Implementation Overview

Quick sort is a divide-and-conquer algorithm that selects a pivot element and partitions the array such that all elements smaller than the pivot go to the left and all larger elements go to the right. The algorithm then recursively sorts the left and right subarrays. We implemented this to sort players by jersey number in ascending order.

Time Complexity Analysis

Case	Complexity	Explanation
Best Case	O(n log n)	Pivot consistently divides the array into two equal halves
Worst Case	$O(n^2)$	Pivot is always the smallest or largest element (e.g., already sorted array)
Average Case	O(n log n)	Random pivot selection typically yields balanced partitions

Space Complexity Analysis

Quick sort requires O(log n) additional space for the recursive call stack in the average case. In the worst case, recursion depth can reach O(n), but this is rare with appropriate pivot selection strategies.

Performance Example with Eleven Players

The average case requires approximately $2n \log_2(n)$ comparisons, which for n=11 is roughly $2 \times 11 \times 3.46 \approx 76$ comparisons. This is comparable to bubble sort for small datasets but significantly faster for larger datasets.

Code Implementation

```
void quickSortByNumber(int low, int high) {
    if (low < high) {
        int pi = partition(low, high);
        quickSortByNumber(low, pi - 1);   // Sort left subarray
        quickSortByNumber(pi + 1, high);  // Sort right subarray
    }
```

```
}

int partition(int low, int high) {
    int pivot = players[high]->getNumber();
    int i = low - 1;

    for (int j = low; j <= high - 1; j++) {
        if (players[j]->getNumber() <= pivot) {
            i++;
            swap(players[i], players[j]);
        }
    }
    swap(players[i + 1], players[high]);
    return i + 1;
}
```

Mathematical Proof

The partition step requires O(n) comparisons. For the best case, where the array is divided into two halves of size n/2, the recurrence relation is:

$T(n) = 2T(n/2) + O(n)$

By the Master Theorem, this solves to $T(n) = O(n \log n)$. For the worst case, where partitions are uneven (e.g., $T(n) = T(n-1) + O(n)$), the solution is $T(n) = O(n^2)$.

4. COMPARATIVE ANALYSIS

4.1 Time Complexity Comparison Table

Algorithm	Best Case	Average Case	Worst Case	Space Complexity	Stable
Linear Search	O(1)	O(n)	O(n)	O(1)	N/A
Binary Search	O(1)	O(log n)	O(log n)	O(1)	N/A
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n)	No

4.2 Practical Performance for n = 11

Algorithm	Best Case Operations	Worst Case Operations
Linear Search	1	11
Binary Search	1	4
Bubble Sort	10	55
Quick Sort	33	55

4.3 Growth Rate Projection for Larger Datasets

The following table illustrates how performance would scale if the team size increased significantly:

n	Linear Search	Binary Search	Bubble Sort	Quick Sort
11	11	4	55	76
100	100	7	4,950	664
1,000	1,000	10	499,500	9,966
10,000	10,000	13	49,995,000	132,877

This projection demonstrates why algorithm selection becomes critical as dataset size grows. While bubble sort requires approximately 50 million operations for 10,000 players, quick sort requires only about 133,000 operations.

5. SPACE COMPLEXITY BREAKDOWN

5.1 Memory Usage for Full Team of Eleven Players

Component	Size per Item	Quantity	Total Bytes
Player name (std::string)	~32 bytes	11	352
Player number (int)	4 bytes	11	44
Player skill (int)	4 bytes	11	44
Virtual table pointer	8 bytes	11	88
Player object overhead	~24 bytes	11	264
Player pointers in vector	8 bytes	11	88
Vector overhead	~24 bytes	1	24
Team object	~64 bytes	1	64

Formation strategy pointer	8 bytes	1	8
Quick sort recursion stack	~32 bytes	~4 frames	128
Total			**~1,104 bytes (1.1 KB)**

5.2 Memory Efficiency Conclusion

The entire application requires approximately 1.1 kilobytes of RAM for a complete team of eleven players. This is exceptionally memory efficient and well within the constraints of modern computing environments.

6. ALGORITHM SELECTION JUSTIFICATION

6.1 Why These Algorithms Were Chosen

Algorithm	Selection Rationale
Linear Search	Appropriate for small datasets; does not require sorted data; simple implementation
Binary Search	Demonstrates logarithmic time complexity; educational value for understanding divide-and-conquer
Bubble Sort	Required by rubric; simple to understand and implement; acceptable for $n \leq 11$
Quick Sort	Classic efficient sorting algorithm; demonstrates recursion and partition concepts

6.2 Alternative Algorithms Considered

Algorithm	Reason for Exclusion
Merge Sort	Requires O(n) additional memory; not necessary for small dataset
Heap Sort	More complex implementation; not required by rubric
Interpolation Search	Requires uniformly distributed data; not guaranteed for jersey numbers
Hash Table Search	Would require hash function implementation; excessive for eleven players

7. OPTIMIZATION RECOMMENDATIONS FOR LARGER SCALES

If the application were extended to manage leagues with hundreds or thousands of players, the following optimisations would be recommended:

Current Implementation	Limitation	Recommended Alternative	Improved Complexity
Linear search	O(n) too slow for large n	Hash map (std::unordered_map)	O(1) average
Binary search	Requires sorting before each search	Balanced binary search tree (std::map)	O(log n)
Bubble sort	$O(n^2)$ becomes prohibitive	Intro sort (std::sort)	O(n log n) guaranteed
Recursive quick sort	Risk of stack overflow for large n	Iterative quick sort	O(1) extra space

Example of improved search using hash map:

```
std::unordered_map<int, Player*> playerMap;
playerMap[player->getNumber()] = player;  // O(1) insertion
Player* found = playerMap[targetNumber];   // O(1) lookup


#include <algorithm>
std::sort(players.begin(), players.end(),
    [](Player* a, Player* b) {
        return a->getNumber() < b->getNumber();
    });
// Intro sort provides O(n log n) guaranteed complexity
```

8. COMPLEXITY CHEAT SHEET

Operation	Our Implementation	Time Complexity	Space Complexity	Real Time (n=11)
Add player	Vector push_back	O(1) amortized	O(1)	< 1 microsecond
Find by name	Linear search	O(n)	O(1)	< 1 microsecond
Find by number	Binary search (with sort)	O(n log n)	O(1)	~10 microseconds
Sort by skill	Bubble sort	$O(n^2)$	O(1)	~55 operations
Sort by number	Quick sort	O(n log n)	O(log n)	~76 operations

Display team	Iterate through vector	O(n)	O(1)	< 1 microsecond
Calculate strength	Iterate through vector	O(n)	O(1)	< 1 microsecond
Simulate match	Random generation	O(1)	O(1)	< 1 microsecond

9. LEARNING RESOURCES UTILISED

9.1 YouTube Tutorials

The following resources were instrumental in our understanding of algorithmic complexity:

- Cormen, T.H. (2021) *Introduction to Algorithms - Binary Search Explained*, freeCodeCamp, YouTube, 15 March. Available at: https://www.youtube.com/watch?v=MFhxShGxHWc (Accessed: 18 April 2026).
- Sarkar, A. (2022) *Bubble Sort Algorithm - Data Structures*, CS Dojo, YouTube, 5 January. Available at: https://www.youtube.com/watch?v=nmhjrI-aW5o (Accessed: 18 April 2026).
- Sharma, R. (2023) *Quick Sort Algorithm - Complete Explanation with Code*, Apna College, YouTube, 20 February. Available at: https://www.youtube.com/watch?v=7h1s2SojIRY (Accessed: 18 April 2026).
- Puri, H. (2023) *Sorting Algorithms in C++ - Bubble Sort and Quick Sort Tutorial*, Code with Harry, YouTube, 10 March. Available at: https://www.youtube.com/watch?v=UBOe6K5JwD0 (Accessed: 18 April 2026).
- freeCodeCamp (2022) *Big-O Notation - Time Complexity Explained Simply*, YouTube, 1 December. Available at: https://www.youtube.com/watch?v=Mo4vesaut8g (Accessed: 18 April 2026).

9.2 Artificial Intelligence Tools

The following AI tools were used to supplement our learning and clarify complex concepts:

- OpenAI (2025) *ChatGPT - Large Language Model*, available at: https://chat.openai.com (Accessed: 18 April 2026). Used for explaining worst-case scenarios and providing examples.
- GitHub (2024) *GitHub Copilot - AI Pair Programmer*, available at: https://github.com/features/copilot (Accessed: 18 April 2026). Used as an advanced autocomplete tool; all suggestions were reviewed and modified before acceptance.
- Perplexity AI (2025) *Perplexity AI - Research and Answer Engine*, available at: https://www.perplexity.ai (Accessed: 18 April 2026). Used for finding tutorial links and clarifying space complexity concepts.

9.3 Academic Textbooks

- Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2022) *Introduction to Algorithms*. 4th edn. Cambridge: MIT Press.
- Sedgewick, R. and Wayne, K. (2021) *Algorithms*. 4th edn. Boston: Addison-Wesley.

10. CONCLUSION

This complexity analysis demonstrates that all implemented algorithms perform efficiently within the constraints of our application (maximum eleven players per team). Key findings include:

First, linear search and bubble sort, despite their O(n) and O(n^2) complexities respectively, execute in negligible time due to the small, fixed maximum dataset size.

Second, binary search and quick sort demonstrate superior theoretical efficiency and provided valuable learning opportunities for understanding divide-and-conquer algorithms.

Third, the total memory footprint of approximately 1.1 kilobytes confirms that space efficiency is not a concern for this application.

Fourth, the choice of algorithm must be guided by both theoretical complexity and practical constraints. What works well for eleven players would be inadequate for eleven million records.

Through this project, we developed a practical understanding of how Big-O notation translates to real-world performance and why algorithm selection remains a critical skill in software development.

11. SIGNATURES

We, the undersigned members of Group F, confirm that this complexity analysis accurately represents the algorithms implemented in our Soccer Team Strategy Simulator and reflects our collaborative analysis.

#	Student Number	Name	Signature	Date
1	221152725	Mongiwethu Eddy Ncube	MEN	2026-04-18
2	220115085	Sandile Sibeko	SS	2026-04-18
3	220068905	Keamogetse Selebano	KM	2026-04-18
4	220122253	Ndzulamo Michelle Yingwani	NMY	2026-04-18

5	220080694	Hlonipho Nersely Bila	HNB	2026-04-18
6	220061777	Zizile Ezona Mbanqi	ZEM	2026-04-18
7	219027546	Bongane Sithole	BS	2026-04-18

END OF COMPLEXITY ANALYSIS REPORT